favorite in one browser instantly updates any other browsers that may be open for that user.

Before we start, we can do some refactoring of our views to make it much easier to incorporate the turbo-rails ActionCable helpers. We have a problem that is general to lots of applications and will probably be similar to issues you might have.

To make sense of the problem, I think it will help to back up a step and look at how our data flow is changing.

In the code as it exists, when we click the Make Favorite button to turn a concert into a favorite, we send a form request to our server, and it sends back a response made up of Turbo Stream flavored-HTML, which Turbo on the client side uses to update the DOM.

In the version we are about to write, we still click the Make Favorite button and still get an HTML request, but after the new Favorite object is created on the server side, the server queues up—and eventually sends—a Turbo Stream message back to the client via ActionCable. Then Turbo client-side updates the DOM based on the Turbo Stream. (We're sending the message using turbo-rails commands that attach to the model, but the general problem still holds even if we sent the message via ActionCable from the controller or from some extra service object.)

What's important to understand about the ActionCable response is that it happens outside the regular Rails request/response cycle. Because of that, the ActionCable broadcast does not have access to global variables or session variables based on the request because it no longer takes place inside the session. Specifically, for our purposes, many of our view partials we've been using to display the schedule depend on the current_user session variable to determine whether the concert is a favorite and whether to display the Edit button.

We need to change the code—any partials that will be called by ActionCable need to have the user passed to them rather than calling current_user inside the